

SSH Terminal

中文维护手册

版本: v0.3.0

GitHub: <https://github.com/leungbzai-png/ssh-terminal>

技术栈: Go + Wails v2 + Vue 3 + TypeScript

生成日期: 2026-06-10

本手册由 Claude Sonnet 4.6 基于 v0.3.0 源码自动生成，供项目维护者参考。

目 录

目录	2
一、项目简介	2
用途	2
当前版本	2
GitHub 地址	2
技术栈	3
二、当前状态 (v0.3.0)	3
已发布内容	3
已知遗留问题 (均为低优先级)	4
三、每次开发前必须阅读	4
第一步: `docs/ROADMAP.md`	4
第二步: `docs/SESSION_STATUS.md`	4
第三步: `docs/AI_HANDOFF.md`	5
第四步: `docs/PROJECT_CONTEXT.md`	5
四、标准开发流程	5
总体步骤	5
分支命名规范	6
关于用 AI 开发	6
五、文档维护规则	6
每完成一个功能, 必须更新这四个文件	7
什么时候更新版本号	7
六、测试检查清单	7
命令一: `go vet ./...`	8
命令二: `go build ./...`	8
命令三: `vue-tsc --noEmit` (含在 npm run build 里)	8
命令四: 完整 Wails 构建 (发版时必须)	8
手工检查 (最少验证项)	9
七、GitHub 发布流程	9
完整操作步骤	9
第一步: 确认代码干净	9
第二步: 提交代码	9

版本: v0.3.0	10
最后更新: 2026-06-10	10
适用人员: 项目作者 (半年后的自己)、AI 协作开发者	10
第三步: 推送到 main	10
第四步: 构建 Windows zip 包	10
第五步: 创建 Git Tag	10
第六步: 在 GitHub 创建 Release	10
第七步: 发布后更新文档	11
八、出问题时怎么排查	11
目录	11
常用排查命令	11
• 项目简介	12
• 当前状态(v0.3.0)	13
• 每次开发前必读	13
• 标准开发流程 (按顺序)	13
• 文档维护规则	14
• 测试检查清单	14
• GitHub 发布流程	14
• AI 接手标准提示词	14
• 出问题时怎么排查	
• v0.4.0 开发建议	
• AI 接手标准提示词	

一、项目简介

用途

SSH Terminal 是一个运行在 Windows 上的轻量级 SSH 终端工具, 主要特点:

- **便携运行**: 不需要安装, 双击 exe 即可用, 数据全部存放在 exe 旁边的 **data/** 文件夹
- **内置 SFTP**: 打开任意 SSH

会话后可以切换到文件浏览器, 支持上传、下载、新建文件夹、重命名、递归删除

- **标签页 + 分屏**: 同时管理多个 SSH 连接, 最多 4 个分屏
- **密码加密**: 存储的密码和密钥口令用 AES-256-GCM 加密, 密钥文件放在本机
- **主机密钥验证**: 严格的 known_hosts 检查, 首次连接时展示指纹让用户确认

当前版本

v0.3.0 (2026-06-10 发布)

GitHub 地址

<https://github.com/leungbzai-png/ssh-terminal>

技术栈

层次	技术	版本
桌面框架	Wails v2 (WebView2 内核)	v2.12.0
后端语言	Go	1.22+
SSH 库	golang.org/x/crypto/ssh	v0.33.0
SFTP 库	github.com/pkg/sftp	v1.13.6
前端框架	Vue 3 + Pinia	3.5.12 / 2.2.4
终端组件	xterm.js	5.5.0
构建工具	Vite + vue-tsc	5.4.10 / 2.1.10
操作系统	Windows 10/11 (仅限)	—

二、当前状态 (v0.3.0)

已发布内容

v0.3.0 已于 2026-06-10 完整发布，包含以下内容：

新功能

- SFTP 面板所有 **confirm()** / **prompt()** 替换为应用内自定义弹窗 (**ConfirmDialog**、**InputDialog**)
- SFTP 支持递归删除非空文件夹 (后端用 **sftp.Client.RemoveAll**，含根路径安全守卫)
- 连接超时可配置 (全局设置，默认 15 秒，范围 5–120 秒)
- GitHub Actions CI 自动化 (每次推送到 main 时跑 Go 编译检查 + 前端构建检查)

已建立的文档体系

文件	作用
`docs/ROADMAP.md`	功能规划和优先级
`docs/SESSION\_STATUS.md`	当前开发状态快照
`docs/AI\_HANDOFF.md`	AI 接手指南
`docs/PROJECT\_CONTEXT.md`	设计决策背景
`docs/RELEASE\_PROCESS.md`	发布操作手册

`docs/QA\_CHECKLIST.md`	手工测试清单
`docs/architecture.md`	架构图和数据流
`CHANGELOG.md`	版本变更记录

已建立的工程体系

- Git 仓库初始化, 已推送到 GitHub
- **.gitignore** 覆盖所有需要忽略的文件
- **build-windows.bat** 一键构建脚本
- GitHub Actions CI (Go + 前端双检查)
- GitHub Releases 含 Windows zip 附件

已知遗留问题 (均为低优先级)

编号	描述	计划版本
KI-03	`known_hosts` 每台主机可能有两条记录	v0.4.0
KI-04	终端波特率硬编码为 14400	v0.4.0
KI-06	`buildAuthForDeploy` 与 `buildAuth` 冲突	v0.4.0
KI-07	`ConfirmQuit()` 中有 80ms 的 `time.Sleep`	v0.4.0
KI-08	重连时不关闭旧的 SSH 连接	v0.4.0

三、每次开发前必须阅读

不管隔了多久, 开始写代码之前必须先读这四个文件, 顺序不能颠倒。

第一步: `docs/ROADMAP.md`

作用: 告诉你接下来要做什么, 以及为什么这么排优先级。

读这个文件你能知道:

- v0.4.0 计划做哪些功能
- 每个功能的预估收益和难度
- 哪些东西明确不做 (非目标)
- 与竞品 (FinalShell、Xshell、Tabby) 的定位差异

特别注意: ROADMAP 里有些功能标了"延期", 开始之前确认还在计划内, 没有被取消。

第二步: `docs/SESSION\_STATUS.md`

作用: 这是项目的"当前快照", 记录着最近一次开发结束时项目处于什么状态。

读这个文件你能知道:

- 当前版本号是多少
- 最新一次 git commit 是什么
- 上次完成了哪些工作
- 还有哪些已知问题没修
- 下一步打算做什么

为什么重要: 如果你隔了三个月没碰这个项目, 或者让 AI 接手, 这个文件是最快让人了解现状的入口。

第三步: `docs/AI\_HANDOFF.md`

作用: 给 AI 或新维护者的技术接手手册。

读这个文件你能知道:

- Wails 框架怎么运作 (Go 与前端的通信机制)
- 数据文件在哪里, 格式是什么
- 加密模型的设计
- SSH 会话的生命周期
- **已知的坑** (不读这个很容易踩坑): 比如 **useTheme**

不能加生命周期钩子、拖拽上传需要两处同时配置等

第四步: `docs/PROJECT\_CONTEXT.md`

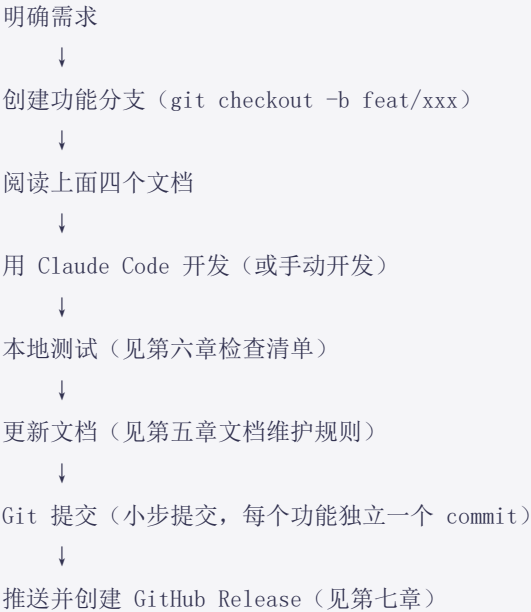
作用: 解释"为什么这么设计", 防止你在不了解背景的情况下改掉某个有意为之的决定。

读这个文件你能知道:

- 为什么选 Wails 而不是 Electron
- 为什么数据放在 exe 旁边而不是 **%APPDATA%**
- 为什么用本地密钥文件加密而不是系统凭证管理
- 为什么用 **v-show** 而不是 **v-if** 显示终端

四、标准开发流程

总体步骤



分支命名规范

类型	格式	示例
新功能	`feat/功能名`	`feat/ssh-config-import`
修复	`fix/问题描述`	`fix/sftp-delete-error`
文档	`docs/内容`	`docs/update-roadmap`
重构	`refactor/模块名`	`refactor/auth-builder`
发布	`release/版本号`	`release/v0.4.0`

关于用 AI 开发

这个项目适合用 Claude Code 或类似工具辅助开发。建议的使用方式：

- 把第十章的"AI 接手标准提示词"发给 AI
- 让 AI 先阅读四个文档，再开始写代码
- 告诉 AI 不要自作主张改架构，只做需求范围内的最小改动
- 每完成一个大功能，要求 AI 同步更新文档

五、文档维护规则

铁律：代码变了，文档必须同步变。不允许文档和代码状态不一致。

每完成一个功能，必须更新这四个文件

`CHANGELOG.md`

- 在文件顶部添加新版本的变更说明
- 格式：## [版本号] - 日期，然后分 Added / Fixed / Changed 三类

`docs/ROADMAP.md`

- 已完成的功能打勾 [x]
- 延期的功能注明原因和新目标版本
- 更新文件顶部的"最后更新"日期

`docs/SESSION\_STATUS.md`

- 更新"当前版本"一栏
- 更新"最新 commit"一栏
- 在"已完成工作"里添加本次完成内容
- 更新"下一步方向"

`docs/AL\_HANDOFF.md`

- 如果有新的 API (Go 方法、前端类型) 要在这里说明
- 如果发现新的坑，加到"已知坑"章节

什么时候更新版本号

版本号遵循 **主版本.次版本.补丁版本** 规则：

改动类型	版本号变化
新功能（向后兼容）	次版本 +1（如 0.3.0 → 0.4.0）
只修 bug	补丁 +1（如 0.3.0 → 0.3.1）
破坏性改动（如 hosts.json 格式变化）	主版本 +1
只改文档/工具	不需要发新版本

版本号需要在三处保持一致：

- **app.go** (**AppInfo()** 函数里)
- **wails.json** (**productVersion** 字段)
- **frontend/package.json** (**version** 字段)

六、测试检查清单

每次发布前，以下命令必须全部通过。

命令一：`go vet ./...`

```
cd E:\Projects\Active\ssh-terminal
go vet ./...
```

作用：Go

官方静态分析工具，检查常见编程错误（错误的格式化字符串、不可达代码、锁使用错误等）。不会执行代码，只做语法和语义检查。

预期结果：没有任何输出，退出码 0。

命令二：`go build ./...`

```
go build ./...
```

作用：编译所有 Go 包，确保没有编译错误。这一步不生成最终 exe，只验证代码能编译通过。

预期结果：没有任何输出，退出码 0。

命令三：`vue-tsc --noEmit`（含在 npm run build 里）

```
cd frontend
npm run build
```

作用：npm run build 会先跑 vue-tsc --noEmit 做 TypeScript 类型检查，然后再用 Vite 打包。如果类型有问题，在类型检查阶段就会报错，不会进行打包。

预期结果：TypeScript 检查通过（0 错误），Vite 打包成功，输出 **dist/** 目录。

命令四：完整 Wails 构建（发版时必须）

```
cd E:\Projects\Active\ssh-terminal
.\build-windows.bat
```

作用：完整构建流程——先跑前端 Vite 打包，再用 Wails 编译 Go + 前端资源合并成一个 exe 文件。这是最接近生产环境的测试。

预期结果：build\bin\ssh-terminal.exe 生成，终端输出 **=== Done ===**。

注意：Vite 的 **emptyOutDir: true** 会在每次构建时清空 **frontend/dist/**，包括 **.gitkeep** 文件。**build-windows.bat** 里已经有对应的恢复逻辑，不需要手动处理。

手工检查（最少验证项）

在跑完上面命令后，手工启动 exe 验证至少这几个场景：

- [] 能打开应用，没有崩溃
- [] 能添加一个 SSH 主机并成功连接
- [] 能打开 SFTP 面板，列出目录
- [] 能在 SFTP 面板新建文件夹（弹出 InputDialog）
- [] 能在 SFTP 面板删除文件（弹出 ConfirmDialog）
- [] 设置里能修改连接超时，保存后重启仍有效

七、GitHub 发布流程

完整操作步骤

第一步：确认代码干净

```
git status
```

应该显示 **nothing to commit, working tree clean**。如果有未提交的改动先处理。

第二步：提交代码

建议按功能分小 commit，不要一次提交几百行：

```
git add <具体文件>
git commit -m "feat: 功能描述"
```

常用 commit 类型前缀：

- **feat:** 新功能
- **fix:** 修复
- **docs:** 文档
- **ci:** CI 流程

- **chore:** 版本号、配置等杂项

第三步：推送到 main

```
git push origin main
```

推送后 GitHub Actions 会自动运行 CI，在 GitHub 仓库的 Actions 页面可以看到结果。等 CI 通过再继续。

第四步：构建 Windows zip 包

```
.\build-windows.bat
```

然后打包：

```
$version = "0.4.0" # 改成实际版本号
$zipPath = "E:\Backup\Releases\ssh-terminal-v$version-windows-amd64.zip"
$tmpDir = "E:\Backup\Releases\pkg_v040"

[System.IO.Directory]::CreateDirectory($tmpDir) | Out-Null
Copy-Item "build\bin\ssh-terminal.exe" "$tmpDir\ssh-terminal.exe"
Copy-Item "README.md" "$tmpDir\README.md"
Copy-Item "LICENSE" "$tmpDir\LICENSE"

Add-Type -AssemblyName System.IO.Compression.FileSystem
[System.IO.Compression.ZipFile]::CreateFromDirectory($tmpDir, $zipPath)
[System.IO.Directory]::Delete($tmpDir, $true)

Write-Output "zip: $((Get-Item $zipPath).Length / 1MB) MB"
```

zip 包内容：**ssh-terminal.exe**、**README.md**、**LICENSE**，三个文件，不多也不少。

第五步：创建 Git Tag

```
git tag -a "v0.4.0" -m "SSH Terminal v0.4.0"
git push origin "v0.4.0"
```

第六步：在 GitHub 创建 Release

先把 Release Notes 写到一个文件，再用 gh 命令上传：

```
gh release create "v0.4.0" `
  "E:\Backup\Releases\ssh-terminal-v0.4.0-windows-amd64.zip" `
  --repo "leungbzhai-png/ssh-terminal" `
  --title "SSH Terminal v0.4.0" `
  --latest `
  --notes-file "E:\Backup\Releases\release-notes-v0.4.0.md"
```

Release Notes 内容建议包含：

- 新功能说明（每条一段）
- 修复内容
- 测试结果表格
- 已知问题列表
- 下载和安装说明

第七步：发布后更新文档

```
# 更新 SESSION_STATUS.md 里的最新 commit hash
git add docs/SESSION_STATUS.md
git commit -m "docs: update session status after v0.4.0 release"
git push origin main
```

八、出问题时怎么排查

常用排查命令

查看当前状态

```
git status
```

告诉你哪些文件改了还没提交。如果有意外的改动，先搞清楚是什么再决定要不要提交。

查看最近提交历史

```
git log --oneline -10
```

看最近 10 条提交，确认自己在哪个版本上。

查看某个文件改了什么

```
git diff app.go
```

或者查看两个版本之间的差异：

```
git diff v0.3.0 v0.4.0 -- app.go
```

查看项目当前状态快照

打开 **docs/SESSION_STATUS.md**，里面记录了：

- 当前版本号
- 最新 commit
- 已知问题
- 下一步计划

查看版本变更记录

打开 **CHANGELOG.md**，按时间倒序记录了每个版本做了什么。

常见问题和解决方法

问题：`go build` 报错找不到包

检查 **go.mod** 里的模块名是否是 **github.com/leungbzai-png/ssh-terminal**，然后运行 **go mod tidy**。

问题：前端 TypeScript 报类型错误

通常是因为修改了 Go 的 API（比如改了方法签名或加了新方法），但没有同步更新 **frontend/src/wails.d.ts**。对照 **app.go** 里的方法补齐类型声明。

问题：设置没有保存

检查 **data/settings.json** 是否存在，以及 **internal/config/config.go** 里的 **Save()** 函数是否正常。可以直接打开 **settings.json** 查看内容是否符合预期。

问题：SSH 连接失败

依次检查：

- 主机地址和端口是否正确
- 用户名是否正确
- 如果用密钥，密钥文件路径是否存在
- 查看 **data/known_hosts** 里有没有这台主机的记录

问题：SFTP 操作失败

SFTP 依赖 SSH 会话，如果 SSH 断了 SFTP 也会失败。先确认 SSH 连接是否正常（终端里能不能输入命令）。

回滚方案

回滚到上一个版本（不影响已推送的历史）

```
# 查看历史找到要回滚的 commit hash
git log --oneline

# 创建一个新 commit 来撤销某次改动（推荐，不破坏历史）
git revert <commit-hash>
git push origin main
```

回滚到某个 tag 版本（本地硬回滚，谨慎操作）

```
git checkout v0.3.0      # 切换到 v0.3.0 的代码查看
git checkout main        # 切回 main
```

如果真的需要强制回退（会丢失历史，极少需要这样做）：

```
# 危险操作，确认后再执行
git reset --hard v0.3.0
git push --force origin main
```

九、v0.4.0 开发建议

建议优先做（按顺序）

1. 从 `~/.ssh/config` 导入主机

用户价值最高，实现难度低。读取用户的 SSH 配置文件，解析 Host 条目，批量导入主机列表。

实现要点：

- 读取 **C:\Users\用户名\.ssh\config**
- 解析 **Host**、**HostName**、**User**、**Port**、**IdentityFile** 字段
- 展示预览列表让用户勾选要导入的条目
- 调用现有的 **hosts.Upsert()** 保存

2. Quick Connect (临时连接, 不保存)

频繁用到不常保存的场景很常见 (比如临时访问某台服务器)。

实现要点:

- 在侧边栏加一个"快速连接"入口
- 弹出一个简单表单 (地址、用户、认证方式)
- 不调用 **hosts.Upsert()**, 直接用临时 host 对象发起 **OpenSession**
- 标签页显示"临时 - 地址"

3. SSH Session Keep-Alive

长时间不操作后 SSH 连接会被服务器断开, 这个问题用户反馈多。

实现要点:

- 在 **hosts.Host** 结构体加 **KeepAliveInterval int** (秒, 0 表示关闭)
- 在 **ssh sess.Manager.Open()** 里起一个定时 goroutine, 定期发空包
- 在主机编辑界面加对应设置项

建议暂缓

ProxyJump / 跳板机支持

- 需要修改 **hosts.json** 格式 (加 **jumpHost** 字段), 属于破坏性改动
- 建议等单元测试覆盖 **config** 和 **hosts** 包之后再动
- 实现也更复杂 (需要链式 SSH dial)

大规模重构

- 目前代码结构清晰, 没有重构的必要
- 不要在没有测试覆盖的情况下重构核心模块 (**cryptox**、**hosts**)
- 如果发现重复代码 (比如 **buildAuth** / **buildAuthForDeploy**), 等出现第三个调用方再考虑提取

关于添加单元测试

v0.4.0 建议开始补测试, 优先级:

- **internal/cryptox** — 纯函数, 最容易, 最重要 (涉及加密)
- **internal/portable** — 纯路径解析, 几分钟能写完
- **internal/config** — JSON 读写的往返测试
- **internal/hosts** — 需要临时目录, 稍复杂一点

十、AI 接手标准提示词

当你需要让 Claude Code 或其他 AI 工具接手这个项目时，把下面这段话直接发给它：

你好，请接手这个项目：SSH Terminal。

项目是一个 Windows 桌面 SSH 客户端，使用 Go + Wails v2 + Vue 3 开发。

GitHub: <https://github.com/leungbzai-png/ssh-terminal>

在开始任何工作之前，请按顺序阅读以下文件：

1. docs/SESSION_STATUS.md ← 了解项目当前状态和最新进展
2. docs/AI_HANDOFF.md ← 了解技术架构和已知的坑
3. docs/ROADMAP.md ← 了解接下来要做什么
4. docs/PROJECT_CONTEXT.md ← 了解设计决策背景

本次任务：[在这里填写具体需求]

工作要求：

- 不要自作主张修改架构，只做需求范围内的最小改动
- 不要修改版本号（除非任务明确要求发版）
- 不要执行 `git commit` / `git push`（除非任务明确包含这一步）
- 不要删除源代码文件
- 每完成一个功能要同步更新对应的文档
- 代码改完后必须跑 `go vet ./...`、`go build ./...` 和前端构建，确认通过

如果发现需求与现有代码有冲突，请停下来说明冲突原因和建议的最小可行方案，不要自行决策大改。

附：常用的接手场景模板

场景一：实现单个新功能

任务：实现 [功能名称]

背景：[为什么要做这个，ROADMAP 里的编号是什么]

具体需求：

- [需求点1]
- [需求点2]

- [需求点3]

完成标准:

- [] go build ./... 通过
- [] 前端构建通过
- [] 手工测试: [描述测试步骤]
- [] 文档已更新 (CHANGELOG、SESSION_STATUS、AI_HANDOFF)

场景二: 修复 Bug

任务: 修复 [问题描述]

复现步骤:

1. [步骤1]
2. [步骤2]
3. 期望结果: [正确行为]
4. 实际结果: [错误行为]

相关文件: [如果知道的话]

不要改其他东西, 只修这个问题。

场景三: 发布新版本

任务: 完成 v[X.Y.Z] 的版本封装和发布

1. 检查 git status 确认 working tree clean
2. 更新版本号 (app.go、wails.json、frontend/package.json) 到 [X.Y.Z]
3. 更新 CHANGELOG.md, 添加 v[X.Y.Z] 的变更说明
4. 更新 docs/ROADMAP.md (标记已完成项)
5. 更新 docs/SESSION_STATUS.md (当前版本、最新 commit、完成内容)
6. 运行全套构建检查 (go vet、go build、前端构建、wails build)
7. 按以下 commit 顺序提交: [列出 commit 列表]
8. 推送 main, 创建并推送 tag v[X.Y.Z]
9. 打包 Windows zip, 创建 GitHub Release, 上传 zip

注意: 不执行 git push 之前先让我确认 commit 列表。

本手册由 Claude Sonnet 4.6 基于 v0.3.0 源码和文档自动生成, 2026-06-10。

下次更新时间建议: v0.4.0 发布后。